

GOLANG LIVING TEXTBOOK // TECHNICAL MONOGRAPH

G O L A N G

Giáo trình Cập nhật Liên tục về Kỹ nghệ Phần mềm và DevOps/SRE

ĐOÀN NGỌC HOÀNG MINH

ĐẶC TẢ XUẤT BẢN THƯƠNG MẠI 2026

Định dạng: ISO A4 (210 × 297 mm) // Khâu chỉ dán gáy keo PUR

Hệ thống chữ: Source Serif 4, Source Sans 3 & JetBrains Mono

Kho lưu trữ mã nguồn mở: <https://github.com/Minhlike/Golang>

Bản quyền © 2026 Đoàn Ngọc Hoàng Minh. Phát hành theo giấy phép học thuật.

MỤC LỤC

PHẦN I: NỀN TẢNG CƠ HỌC & BỘ NHỚ GO

Dẫn nhập	Trước khi viết dòng Go đầu tiên	3
Tổng quan	Mở cửa vào Go: Cài đặt và Tư duy Thực chiến	8
Chương 01	Đọc và viết một chương trình Go	14
Chương 02	Khi một bản sao vẫn chia sẻ dữ liệu (Pointer, Slice Header)	35
Chương 03	Mô hình dữ liệu và trách nhiệm thay đổi	45
Chương 04	Biên lỗi: để caller quyết định	68
Chương 05	Package là ranh giới	81

PHẦN II: TƯƠNG TRANH & ĐỘ ỔN ĐỊNH HỆ THỐNG

Chương 06	Thay đổi không sợ hãi (Testing, Benchmarking, Fuzzing)	94
Chương 07	Dữ liệu đi vào và đi ra (io.Reader, Stream Buffering)	114
Chương 08	Một race bắt đầu từ đâu (Memory Model, Race Detector)	124
Chương 09	Dòng công việc có áp suất (Channel Rendezvous, Buffered Queue)	132
Chương 10	Khi chương trình chậm hoặc phình (pprof, Escape Analysis)	140
Chương 11	Lần theo một request HTTP (Routing, Middleware Chain)	153
Chương 12	Một service sống và tắt thế nào (Graceful Shutdown, Context)	160

PHẦN III: DỊCH VỤ PHÂN TÁN & CƠ SỞ DỮ LIỆU

Chương 13	Một thay đổi hoặc không có gì (SQL ACID, Idempotency)	170
Chương 14	Khi kiểu trở thành dữ liệu (reflect, Generic, Type Parameter)	186
Chương 15	Từ incident đến công cụ (CLI, Automation, Diagnostics)	198
Chương 16	Thấy được hệ thống (Prometheus Metrics, OpenTelemetry)	207
Chương 17	Đóng gói và điều phối (Containerization, Multi-stage Build)	218

PHẦN IV: KUBERNETES OPERATOR & eBPF NÂNG CAO

Chương 21	Vòng lặp điều hòa và Controller Pattern	277
Chương 22	Từ watch đến một controller Kubernetes thật	287
Chương 23	Từ controller đến operator hoàn chỉnh (CRD, Reconciler)	305
Chương 27	Quan sát Linux từ kernel bằng eBPF và Go (Cilium, Tracepoint)	379

BACK MATTER // TÀI LIỆU TRA CỨU

Atlas	Hồ sơ kiến trúc 50 thư viện DevOps & Cloud hàng đầu	429
Phụ lục A	Atlas Lỗi Thực Chiến: Chẩn đoán & Khắc phục nhanh	484

Đọc và viết một chương trình Go

MỤC TIÊU SỰ PHẠM VÀ RÀNG BUỘC KỸ NGHỆ // LEARNING OBJECTIVES

Chương này dẫn lối kỹ sư tiếp cận Go không phải như một cú pháp bề mặt, mà như một mô hình cơ học: cách trình biên dịch phân bổ biến trên ngăn xếp, cấu trúc của tệp thực thi ELF sau khi liên kết tĩnh, và nguyên lý điều hòa vòng đời goroutine thông qua runtime footprint tối thiểu. Sau khi hoàn thành chương, người học có thể tự giải thích cội nguồn của mọi quyết định cấp phát bộ nhớ và kiểm chứng mã bằng các công cụ đo lường tiêu chuẩn.

Mỗi ngôn ngữ lập trình đều mang theo một giả định ngầm về cách thế giới vận hành. Với C, đó là bộ nhớ vật lý thẳng đuột với con trỏ tự do. Với Java, đó là một máy ảo trừu tượng quản lý đối tượng qua tham chiếu gián tiếp. Với Go, thế giới được mô hình hóa qua hai trục cơ học vững chắc: giá trị thực thi tức thời (value semantics) và sự đồng bộ hóa qua giao tiếp (channel rendezvous).

Khi một kỹ sư viết dòng lệnh khai báo một cấu trúc struct, câu hỏi đầu tiên xuất hiện trong đầu không nên là cú pháp có ngắn gọn không, mà là: biến này sẽ tồn tại ở đâu trong không gian địa chỉ 64-bit của tiến trình Linux? Nó nằm trên frame của goroutine stack hay bị ép thoát ra heap do escape analysis? Sự khác biệt giữa 0 lần cấp phát (zero-allocation) và 1 lần cấp phát heap chính là lần ranh giữa một dịch vụ vi mô chịu tải 100.000 requests/giây và một dịch vụ sụp đổ vì tail-latency của Garbage Collector.

1.1. Cấu trúc một tệp mã nguồn và ranh giới biên dịch

Mỗi tệp Go bắt đầu bằng mệnh đề package. Package không chỉ là một không gian tên (namespace) logic nhằm tránh đụng độ định danh, mà là ranh giới biên dịch nguyên tử (atomic compilation boundary) của toolchain. Không có khái niệm vòng tròn phụ thuộc (circular dependency) được phép tồn tại: trình biên dịch Go áp đặt đồ thị có hướng không chu trình (DAG) ở mức độ kiến trúc gói.

MẪU BỐ CỤC ĐỐI XỨNG QUA GÁY (MIRRORED FACING SPREADS)

Mình họa quy chuẩn Header/Footer chạy trang (Running Heads & Folios) theo lề sách mở đôi.

48	GOLANG – GIÁO TRÌNH KỸ NGHỆ PHẦN MỀM VÀ DEVOPS/SRE	[TRANG CHẴN / VERSO]
----	--	-------------------------

Đặc tả Lề Trang Chẵn (Verso): Lề ngoài bên trái 18.0 mm (chứa số trang 48); lề trong bên phải 24.0 mm (hướng vào khe gáy sách). Khoảng cách 24 mm đảm bảo chữ không bị che khuất khi mở sách.

QUY TẮC BỘ NHỚ // GO RUNTIME SPECIFICATION:

Mọi biến truyền vào hàm trong Go mặc định đều là truyền theo giá trị (pass-by-value). Khi bạn truyền một slice header, Go chỉ sao chép con trỏ mảng cơ sở, độ dài len và sức chứa cap (24 byte). Dữ liệu phần tử không hề bị nhân bản.

[TRANG LẺ / RECTO]	CHƯƠNG 03 – MÔ HÌNH DỮ LIỆU VÀ TRÁCH NHIỆM THAY ĐỔI	49
-----------------------	---	----

Đặc tả Lề Trang Lẻ (Recto): Lề trong bên trái 24.0 mm (hướng vào khe gáy sách); lề ngoài bên phải 18.0 mm (chứa số trang 49). Tiêu đề chương chạy ở đầu trang lẻ giúp người đọc lật nhanh để định vị nội dung.

Một quy chuẩn bất di bất dịch của nghệ thuật chế bản xuất bản là: Trang mở đầu chương (Chapter Opener) tuyệt đối không bao giờ được có Running Head trên đỉnh. Trên trang mở đầu chương, số trang sẽ hoặc là ẩn đi, hoặc là đưa xuống đáy trang (Drop Folio). Running Head chỉ bắt đầu xuất hiện từ trang thứ hai của chương trở đi.